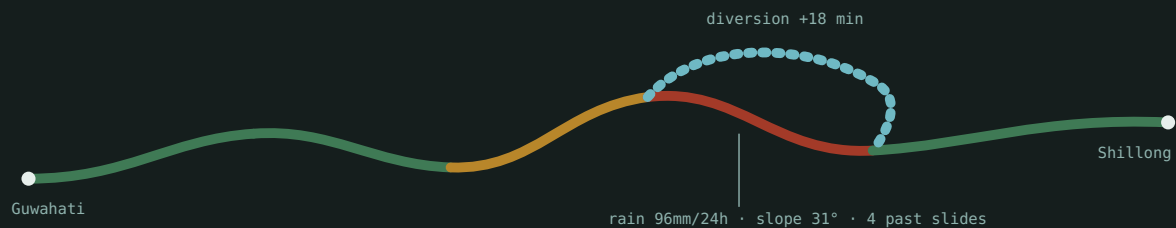


Build manual for the NER hazard-aware logistics prototype

Everything below assumes one goal: a working demo that a judge can drive with their own hands in eight minutes. Not a product. Nine modules, in build order, with the parts that are safe to fake marked clearly.



The whole product in one picture: a road graph whose edge weights move with the weather.

9

modules

3

services in compose

1

corridor for the demo

0

live SAR processing

Modules

- 0 · Scope & the cut list
- 1 · Data acquisition
- 2 · PostGIS schema
- 3 · Road graph
- 4 · Risk scoring engine
- 5 · Hazard-aware routing
- 6 · Landslide precursors
- 7 · FastAPI backend
- 8 · React + MapLibre client
- 9 · Offline PWA, SOS, truck bays

Where to build

- Local environment
- Repo layout
- Hosting & demo-day setup
- Build order
- Demo script

MOD 0 Scope, and the things you must refuse to build

Decide what "prototype" means before writing a line of code.

Your idea slide lists six solution pillars. A hackathon prototype that attempts six pillars ends up with six broken ones. Build **three** end to end, wire the other three as thin but honest surfaces, and say so out loud in the presentation — judges reward a team that knows the difference between shipped and stubbed.

Pillar	Prototype depth	Why
Hazard-aware routing	● Build fully	This is the demo. It must be real and interactive.
Risk scoring + alerts	● Build fully	Feeds routing; explains itself on screen.
Offline PWA	● Build fully	Cheap to build, huge differentiator for NER. Demo by turning off wifi.
Trucker services + SOS	● Build thin	Real map layer + working SOS payload; bay availability from seeded data.
Analytics for planning	● Build thin	One heatmap + one chart from historical table. Half a day.
InSAR landslide precursors	● Pre-compute, don't run live	Sentinel-1 interferometry is a multi-week pipeline. See Module 6.

Cut list — do not build these. User accounts and JWT auth (use a header-based fake driver ID). Payments. Native Android app. Real-time GPS fleet tracking. Admin CRUD screens. A chatbot. Kubernetes. Multilingual UI beyond one extra language toggle on two screens. Each of these has eaten a hackathon team alive and none of them is what SIH26002 is asking about.

Pin the geography

Pick one corridor and make it excellent. Recommended: **NH-6 Guwahati → Shillong → Silchar**, roughly 350 km, which covers the Ri-Bhoi truck bays your own research cites, the Meghalaya ghat sections, and the Barak valley flood plain. One corridor means you can hand-verify every risk score, and a judge who knows the region will recognise the places.

Deliverable: a one-page `SCOPE.md` in the repo listing the three deep pillars, the corridor bbox, and the cut list. Everyone on the team reads it before starting.

MOD 1 Data acquisition

Get every byte you need onto disk once, so the demo never depends on someone else's uptime.

Split sources into **static** (downloaded once, versioned into a data folder) and **live** (polled on a schedule, cached in your own DB). Nothing in the demo path should call a third-party API synchronously.

Static sources

What	Source	Format / size
Road network	Geofabrik India extract, or Overpass query for your bbox	.osm.pbf, ~100 MB for NE zone
Elevation (slope)	NASA SRTM 30 m via OpenTopography, or Copernicus DEM	GeoTIFF, 1° tiles
Landslide susceptibility	GSI / Bhuvan NRSC landslide inventory layers	WMS or shapefile
Rainfall climatology	Open-Meteo ERA5 archive API (free, no key)	JSON, hourly, 1940–now
Historical blockades	Manually compiled CSV from news + PIB releases	CSV, 80–150 rows is plenty
Truck bays / facilities	OSM tags <code>highway=rest_area</code> , <code>amenity=fuel</code> , plus hand-added NH-6 bays	GeoJSON

The historical blockade CSV is your secret weapon. Spend one afternoon, with two people, reading news archives and PIB releases for landslides and floods on your corridor over the last five years. Columns: `date`, `lat`, `lon`, `type`, `road_ref`, `duration_hours`, `source_url`. Hand-built, cited, regional — this is the thing that makes the risk model look informed rather than invented, and no other team will have it.

Live sources, and the fetch discipline

Open-Meteo is the backbone: no API key, generous limits, and it returns forecast and archive through the same shape. IMD and CWC pages are HTML that changes without warning — treat them as best-effort enrichment, never as a dependency.

```
# ingest/weather.py – one call covers the whole corridor
import httpx, datetime as dt

OPEN_METEO = "https://api.open-meteo.com/v1/forecast"

async def fetch_rain(points):
    """points: list of (id, lat, lon). Open-Meteo accepts comma lists."""
    params = {
        "latitude": ",".join(str(p[1]) for p in points),
        "longitude": ",".join(str(p[2]) for p in points),
        "hourly": "precipitation,visibility",
        "past_days": 3, "forecast_days": 2, "timezone": "Asia/Kolkata",
    }
    async with httpx.AsyncClient(timeout=30) as c:
        r = await c.get(OPEN_METEO, params=params)
        r.raise_for_status()
        return r.json()
```

Sample the corridor every ~10 km rather than per road segment — weather does not vary meaningfully below that and you will blow your request budget. Store each fetch in a `weather_obs` table with a timestamp; the risk engine reads the DB, never the network.

Deliverable: `make seed` populates a local DB from `/data` with zero network calls, and `make refresh` pulls fresh weather. The demo works with the network unplugged.

MOD 2 PostGIS schema

A schema small enough to hold in your head, spatial enough to do the work in SQL.

PostgreSQL 16 + PostGIS 3.4. Six tables. Resist adding a seventh.

```

CREATE EXTENSION IF NOT EXISTS postgis;

-- one row per directed road segment from the OSM graph
CREATE TABLE segment (
  id          bigserial PRIMARY KEY,
  osm_way_id  bigint,
  u_node      bigint NOT NULL,
  v_node      bigint NOT NULL,
  road_ref    text,          -- 'NH-6'
  highway     text,          -- trunk / primary / ...
  length_m    double precision NOT NULL,
  free_speed_kph smallint NOT NULL DEFAULT 40,
  mean_slope_deg real,       -- from DEM, Module 3
  susceptibility real,       -- 0..1 from GSI raster
  geom        geometry(LineString, 4326) NOT NULL
);
CREATE INDEX ON segment USING GIST (geom);
CREATE INDEX ON segment (u_node);

-- rolling weather, one row per sample point per hour
CREATE TABLE weather_obs (
  id bigserial PRIMARY KEY,
  point_id int, ts timestamptz NOT NULL,
  rain_mm real, rain_24h_mm real, visibility_m real,
  geom geometry(Point, 4326)
);

-- hand-compiled history: the credibility layer
CREATE TABLE incident (
  id bigserial PRIMARY KEY,
  occurred_on date, kind text,      -- landslide | flood | blockade
  duration_hours real, source_url text,
  geom geometry(Point, 4326)
);

-- what the risk engine writes; what the router reads
CREATE TABLE segment_risk (
  segment_id bigint REFERENCES segment(id),
  computed_at timestamptz NOT NULL DEFAULT now(),
  score real NOT NULL,             -- 0..100
  band text NOT NULL,              -- safe | caution | hazard
  factors jsonb NOT NULL,          -- the explanation, verbatim
  PRIMARY KEY (segment_id, computed_at)
);

CREATE TABLE facility (
  id bigserial PRIMARY KEY,
  name text, kind text,            -- truck_bay | fuel | food | medical | toilet
  capacity int, attrs jsonb,
  geom geometry(Point, 4326)
);

-- driver-submitted ground truth + SOS
CREATE TABLE report (
  id bigserial PRIMARY KEY,
  created_at timestamptz DEFAULT now(),
  device_id text, kind text,       -- blocked | slide | water | sos
  note text, confirmed_count int DEFAULT 0,
  geom geometry(Point, 4326)
);

```

Two design choices worth defending to a judge: `segment_risk.factors` is a JSONB blob holding every input that produced the score, so the UI can explain a red segment without a second query; and `report` carries `confirmed_count` so driver reports gain weight when several devices agree, which is the cheapest possible crowd-verification.

Deliverable: `alembic upgrade head` builds this from scratch, and `psql -f seed.sql` fills it in under a minute.

MOD 3 Building the road graph

Turn OSM into a routable graph whose edges carry terrain.

OSMnx does the download and topology cleaning. Do this once, offline, and commit the result as a GraphML file plus a SQL dump — never at request time.

```
# build/graph.py
import osmnx as ox, rasterio, numpy as np
from shapely.geometry import LineString

ox.settings.useful_tags_way += ["ref", "bridge", "tunnel"]

BBOX = (26.60, 24.70, 93.30, 90.90) # N, S, E, W – corridor envelope

G = ox.graph_from_bbox(*BBOX, network_type="drive", simplify=True)
G = ox.truncate.largest_component(G, strongly=True)

# attach mean slope from the DEM to every edge
dem = rasterio.open("data/dem/srtm_ner.tif")

def mean_slope(line: LineString) -> float:
    pts = [line.interpolate(d, normalized=True) for d in np.linspace(0, 1, 8)]
    z = np.array([next(dem.sample([(p.x, p.y)]))[0] for p in pts], dtype=float)
    seg_len = line.length * 111_000 / 7 # deg -> m, per sample gap
    return float(np.degrees(np.arctan(np.abs(np.diff(z)).mean() / max(seg_len, 1))))

for u, v, k, d in G.edges(keys=True, data=True):
    d["mean_slope_deg"] = mean_slope(d["geometry"]) if "geometry" in d else 0.0

ox.save_graphml(G, "data/graph/ner_drive.graphml")
```

Loading the graph at runtime

Load the GraphML into memory once at FastAPI startup, in the lifespan handler, and keep it as a module-level `networkx.MultiDiGraph`. A 350 km corridor is maybe 40k edges — tens of megabytes, instant Dijkstra. Do not rebuild per request, and do not put NetworkX behind an ORM.

The trap: `graph_from_bbox` across all eight NE states returns a graph large enough to make every routing call sluggish and your laptop hot during the demo. Stay inside the corridor envelope. If a judge asks about scale, the answer is that the same pipeline runs per-corridor and the API is stateless behind a graph cache — which is true, and is a better answer than a slow demo.

Deliverable: `segment` table populated, and a script that routes Guwahati → Shillong on plain distance in under 100 ms.

MOD 4 The risk scoring engine

A number between 0 and 100 per segment, and a sentence explaining it.

Two layers, and you need both for different reasons. The **transparent layer** is a weighted index a judge can follow on a slide. The **learned layer** is a gradient-boosted classifier trained on your incident CSV that proves the AI claim. Ship the transparent one as the default and let the learned one adjust it — this ordering also means a bad model can never break the demo.

Layer A — transparent hazard index

```
# risk/index.py
from dataclasses import dataclass, asdict

WEIGHTS = {"rain": .34, "slope": .22, "susceptibility": .18,
           "history": .16, "reports": .10}

def norm(x, lo, hi):
    return max(0.0, min(1.0, (x - lo) / (hi - lo)))

@dataclass
class Factors:
    rain: float; slope: float; susceptibility: float
    history: float; reports: float

def score_segment(seg, wx, hist_count, active_reports) -> tuple[float, dict]:
    f = Factors(
        rain          = norm(wx.rain_24h_mm, 10, 120),      # IMD heavy-rain thresholds
        slope         = norm(seg.mean_slope_deg, 8, 35),
        susceptibility = seg.susceptibility or 0.0,
        history        = norm(hist_count, 0, 6),
        reports        = norm(active_reports, 0, 3),
    )
    raw = sum(WEIGHTS[k] * v for k, v in asdict(f).items())
    return round(raw * 100, 1), asdict(f)

def band(score):
    return "hazard" if score >= 62 else "caution" if score >= 34 else "safe"
```

Every constant here should trace to something citable. 120 mm/24 h is IMD's heavy-rainfall boundary. Slope above roughly 30° is where the landslide literature for the Himalayan foothills puts the sharp rise. Write those citations as comments — judges ask where numbers came from, and "we tuned it until the map looked right" is the wrong answer.

Layer B — the learned adjustment

Build a training table from `incident`: each historical incident is a positive at its segment on its date; sample negatives from the same segments on dry dates and from nearby segments on the same dates. Features = the same five factors plus month and a 72-hour antecedent rainfall sum. Then:

```
from xgboost import XGBClassifier
from sklearn.model_selection import GroupKFold
from sklearn.metrics import roc_auc_score

model = XGBClassifier(n_estimators=250, max_depth=4, learning_rate=.06,
                     subsample=.8, scale_pos_weight=neg/pos, eval_metric="auc")
# group by segment so the same road can't appear in train and test
cv = GroupKFold(n_splits=5)
# ... fit, report mean AUC, joblib.dump(model, "models/risk_xgb.joblib")

def adjusted(base_score, prob):
    """Model nudges the transparent index by at most 15 points."""
    return max(0.0, min(100.0, base_score + (prob - 0.5) * 30))
```

Be honest about the dataset size. With ~120 incidents you have a small-sample model, and a judge with an ML background will know it. Say it first: "the index is the decision-maker, the model is a learned correction, and here is the AUC with segment-grouped cross-validation." That sentence buys more credit than an inflated accuracy claim.

Recompute loop

A single APScheduler job inside the API process, every 15 minutes: pull latest `weather_obs`, count incidents within 2 km of each segment (precompute this once — it does not change), count unexpired reports, write a fresh `segment_risk` row per segment. 40k segments is a couple of seconds with a bulk insert.

Deliverable: GET `/risk/tiles` returns GeoJSON of the corridor coloured by band, and every feature carries its `factors` object.

MOD 5 Hazard-aware routing

Three routes from the same graph, differing only in how much they fear risk.

This is the single most important piece of code in the project, and it is about twenty lines. The trick is that "safest / fastest / balanced" is one parameter, not three algorithms.

```
# routing/engine.py
import networkx as nx

PROFILES = {"fastest": 0.0, "balanced": 1.2, "safest": 4.0}

def edge_cost(alpha, risk_by_seg):
    def w(u, v, data):
        base_min = data["length"] / 1000 / max(data.get("speed_kph", 40), 5) * 60
        r = risk_by_seg.get(data["segment_id"], 0.0) / 100.0
        if r >= 0.92:
            # reported blocked: hard exclude
            return None
        return base_min * (1 + alpha * r ** 2) # squared: mild risk barely penalised
    return w

def route(G, src, dst, profile, risk_by_seg):
    w = edge_cost(PROFILES[profile], risk_by_seg)
    path = nx.shortest_path(G, src, dst, weight=w)
    return summarise(G, path, risk_by_seg)
```

Two details that make it feel finished:

- **Squaring the risk term** means a corridor of mildly damp road doesn't get abandoned, while one genuinely dangerous ghat section dominates. Linear penalties produce silly diversions and reviewers notice.
- **Returning `None` from the weight function** is how NetworkX removes an edge, which is exactly the semantics you want for a confirmed blockade. No graph mutation, no copies.

What the route response must contain

Do not return a bare polyline. Return the explanation, because the explanation is the product:

```
{
  "profile": "safest",
  "distance_km": 112.4, "duration_min": 168,
  "delta_vs_fastest": {"distance_km": +9.1, "duration_min": +18},
  "max_risk": 41.2, "mean_risk": 18.7,
  "geometry": {"type": "LineString", ...},
  "segments": [{"id": 8823, "band": "caution", "score": 41.2,
    "factors": {"rain": .71, "slope": .48, ...}}],
  "advisories": [
    {"at_km": 61, "level": "caution",
      "text": "Heavy rain past 24h on a steep section. Reduce speed; 4 slides recorded here since 2021."},
    {"at_km": 88, "level": "info",
      "text": "Night fog likely after 21:00. Truck bay with facilities in 6 km."}
  ]
}
```

Generate advisory text from templates keyed on the dominant factor in `factors` — no LLM needed, fully deterministic, translatable, and it works offline. If you want a language toggle, translate the ~12 templates into Assamese and Hindi and you are done.

Deliverable: the three profiles visibly diverge on the map when you raise rainfall on a mid-corridor segment.

MOD 6 Landslide precursors — the honest version

Demonstrate the capability without pretending to have built an InSAR pipeline in a week.

Read this before touching Sentinel-1. A working InSAR deformation pipeline means SNAP or ISCE2, coregistration, interferogram stacks, atmospheric correction, and — in the NER specifically — severe coherence loss under dense forest, which your own slide already acknowledges. Teams that start this in a hackathon finish with nothing else done. Do not do it live.

What to build instead

1. **Pre-compute one small area, offline, before the event.** Pick two Sentinel-1 SLC scenes over a known slide site on your corridor, process a single interferogram pair in SNAP, and export a deformation raster. One pair. One site. It takes a day and gives you a genuine artefact.
2. **Ship a deformation layer in the DB** as a raster or a polygon set with `displacement_mm_per_year`, and feed it into `segment.susceptibility` as a bounded boost. The routing engine then demonstrably reacts to satellite data.
3. **Build the detection interface for real** — a time-series anomaly detector over displacement plus antecedent rainfall, using a rolling z-score or an IsolationForest. This is genuine code, runs in milliseconds, and is the part you can actually defend.

```
# precursor/detect.py – runs over any time series, real or seeded
import numpy as np

def rolling_z(series, window=30):
    s = np.asarray(series, dtype=float)
    mu = np.convolve(s, np.ones(window)/window, mode="same")
    sd = np.sqrt(np.convolve((s-mu)**2, np.ones(window)/window, mode="same")) + 1e-6
    return (s - mu) / sd

def precursor_alert(displacement, rain_72h, z_thresh=2.5, rain_thresh=80):
    z = rolling_z(displacement)
    return [
        {"index": i, "z": float(z[i]), "rain_72h": float(rain_72h[i]),
         "level": "high" if rain_72h[i] > rain_thresh else "watch"}
        for i in range(len(z)) if z[i] > z_thresh
    ]
```

In the presentation, phrase it exactly as it is: "we process Sentinel-1 offline into a deformation layer, and the platform consumes that layer plus rainfall through an anomaly detector. In production this is a scheduled processing chain; in the prototype we pre-computed one corridor site." That is a strong answer. An implied claim of live global InSAR is a weak one that collapses under one question.

Deliverable: one real deformation overlay on the map, and an alert panel that fires when displacement anomaly meets heavy rain.

MOD 7 The FastAPI backend

Twelve endpoints, one process, no auth.

```
api/
main.py          # app, lifespan loads graph + model into app.state
deps.py          # db session, graph handle
routers/
  routes.py      # POST /route, GET /route/alternatives
  risk.py        # GET /risk/segments, GET /risk/heatmap
  facilities.py  # GET /facilities, GET /facilities/{id}/availability
  reports.py     # POST /reports, POST /reports/sos, GET /reports/nearby
  analytics.py   # GET /analytics/hotspots, /analytics/seasonal
  sync.py        # POST /sync/batch <- the offline queue drain
services/
models/          # SQLAlchemy 2.0 + GeoAlchemy2
schemas/         # Pydantic v2
```

The lifespan pattern

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
import osmnx as ox, joblib

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.graph = ox.load_graphml("data/graph/ner_drive.graphml")
    app.state.model = joblib.load("models/risk_xgb.joblib")
    app.state.risk_cache = {} # segment_id -> score, refreshed by scheduler
    scheduler.start()
    yield
    scheduler.shutdown()

app = FastAPI(title="NER Logistics Intelligence", lifespan=lifespan)
```


The sync endpoint, which most teams forget

The offline client will queue reports and replay them. Make the endpoint idempotent with a client-generated UUID, accept a batch, and return per-item results so a partial failure doesn't lose the rest:

```
POST /sync/batch
{"device_id": "tr-4471", "items": [
  {"client_id": "a3f1-...", "kind": "blocked", "lat": 25.71, "lon": 91.88,
   "created_at": "2026-09-17T06:12:00+05:30", "note": "slide debris, one lane"}
]}
-- 200 ->
{"results": [{"client_id": "a3f1-...", "status": "accepted", "server_id": 91}]}
```

Since there is no auth, put a soft rate limit on `POST` endpoints (slowapi, 30/min per device_id) so a demo laptop can't be spammed, and keep an `X-Device-Id` header as the only identity. Document that production would use OTP-based driver auth — one line in the README covers it.

Deliverable: `/docs` renders every endpoint with example payloads. That auto-generated OpenAPI page is worth showing to judges for ten seconds.

MOD 8 The React + MapLibre client

Four screens. A driver could use it while a judge watches.

Screen	Contains	Built with
Route	Origin/destination, three profile cards with time and risk deltas, risk-coloured line, advisory list	MapLibre GL JS + a bottom sheet
Corridor	Full corridor risk layer, incident history pins, deformation overlay toggle	Vector source from <code>/risk/segments</code>
Stops	Truck bays, fuel, food, medical along the active route with distance-ahead	Symbol layer + list
Report / SOS	Four big report buttons, one held-to-fire SOS	Plain components, offline-queued

Map setup that will not embarrass you

```
// map/risk-layer.ts
map.addSource("risk", { type: "geojson", data: "/api/risk/segments?bbox=..." });
map.addLayer({
  id: "risk-line", type: "line", source: "risk",
  paint: {
    "line-width": ["interpolate", ["linear"], ["zoom"], 7, 1.5, 13, 6],
    "line-color": ["match", ["get", "band"],
      "hazard", "#A33A28", "caution", "#B8862A", "#3F7A55"],
    "line-opacity": 0.9
  }
});
```

Basemap: use a free MapTiler or Stadia style key for development, but for the demo pre-generate a **PMTiles** archive of your corridor and serve it from your own static host. It is a single file, works offline, needs no key, and removes the one dependency that would make your demo fail on venue wifi.

Make the explanation visible. Tapping a red segment should open a small panel showing the five factor bars from `factors`. This single interaction is what separates "a map with colours" from "an intelligence platform" in a judge's memory. Build it before you build anything else on the frontend.

Deliverable: route → profiles → tap a red segment → see why. Under five taps, on a phone-sized viewport.

MOD 9 Offline PWA, SOS, and truck bays

The three features that make this feel built for the Northeast rather than for a demo.

Offline, in three layers

1. **App shell** — Workbox precache of the built assets. Free with `vite-plugin-pwa`.
2. **Map + data** — on "Download this corridor", fetch the PMTiles slice, the risk GeoJSON, and the facilities GeoJSON into IndexedDB (use `idb`). Show a real size figure and a progress bar; roughly 20–40 MB for 350 km.
3. **Write queue** — every report and SOS goes into an IndexedDB outbox first and to the network second. Register a Background Sync tag; on failure it drains when connectivity returns, with a plain fallback timer for browsers without Background Sync.

```
// sw.js — outbox drain
self.addEventListener("sync", (e) => {
  if (e.tag === "outbox") e.waitUntil(drainOutbox());
});

async function drainOutbox() {
  const items = await idbGetAll("outbox");
  if (!items.length) return;
  const res = await fetch("/api/sync/batch", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ device_id: DEVICE_ID, items })
  });
  if (!res.ok) throw new Error("retry later"); // keeps the sync registered
  const { results } = await res.json();
  await idbDeleteMany("outbox", results.filter(r => r.status === "accepted")
    .map(r => r.client_id));
}
```

SOS

Hold-to-fire for 2 seconds so it can't trigger in a pocket. The payload carries coordinates, last known route, device ID, timestamp, and the nearest three facilities computed on-device from cached data so it is useful even when it queues. On the server, an SOS report writes to `report` with `kind='sos'` and appears immediately on a separate authority-facing map view. Add an audible confirmation and a visible state — "Sent" vs "Queued, will send when online" — because ambiguity in an emergency control is a design failure.

Truck bay availability

Honest version: seed each bay with a capacity and simulate occupancy from a time-of-day curve plus reports from drivers who check in. Label it in the UI as **predicted** with a confidence and a last-updated timestamp. A prediction with a visible timestamp reads as engineering; a fake live number that a judge probes reads as a lie.

Deliverable: put the laptop in airplane mode mid-demo, plan a cached route, file a report, come back online, watch it sync. This moment wins rounds.

WHERE Local environment

Everyone on the team gets an identical stack in one command.

```
# docker-compose.yml
services:
  db:
    image: postgis/postgis:16-3.4
    environment: [POSTGRES_PASSWORD=dev, POSTGRES_DB=ner]
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s

  api:
    build: ./api
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
    environment: [DATABASE_URL=postgresql+psycopg://postgres:dev@db:5432/ner]
    volumes: ["/api:/app", "/data:/data:ro", "/models:/models:ro"]
    ports: ["8000:8000"]
    depends_on: { db: { condition: service_healthy } }

  web:
    build: ./web
    command: npm run dev -- --host
    volumes: ["/web:/app", "/app/node_modules"]
    ports: ["5173:5173"]

volumes: { pgdata: }
```

Python 3.12, `uv` for dependency resolution (dramatically faster than pip when five people are rebuilding images), Node 20 for the frontend. Pin everything: a hackathon loses hours to one machine resolving a different `rasterio`.

The heavy geo libraries. `rasterio`, `pysheds`, `GDAL` and friends are painful to install natively on Windows and on Apple Silicon. Do all geo processing inside the container, or in a conda/mamba environment — never with bare pip on the host. Decide this on day one, in writing, or you will spend an evening on someone's laptop instead of on the product.

WHERE Repository layout

```
ner-logistics/
SCOPE.md          # module 0 output – the contract with yourselves
docker-compose.yml
Makefile          # up / seed / refresh / test / demo-reset
data/             # git-lfs or a shared drive link; NOT plain git
  osm/ dem/ incidents.csv facilities.geojson
  graph/ner_drive.graphml
  tiles/ner_corridor.pmtiles
build/            # one-off offline scripts (graph, DEM, SAR export)
api/              # module 7
models/          # joblib artefacts, versioned by date
web/             # module 8 + 9
docs/
  architecture.png # the diagram for your slide
  demo-script.md
```

Branching: `main` stays demo-ready at all times. Feature branches, small PRs, and a rule that nobody merges anything on the final evening that hasn't been demoed end to end on someone else's machine. Tag a `demo-ready` release the night before and be prepared to present from that tag if the last commits go wrong.

WHERE Hosting, and the demo-day plan

Cloud, for the judges' phones

Piece	Put it on	Notes
Frontend (PWA)	Cloudflare Pages or Vercel	Free, HTTPS by default — required for service workers and geolocation
API	Render, Railway, or Fly.io	Pick a region near India; cold starts on free tiers will bite, so keep a paid \$5 instance for demo week
Postgres + PostGIS	Supabase or Neon	PostGIS available on both free tiers
PMTiles + static data	Cloudflare R2 with a range-request-capable URL	No egress fees; PMTiles is a single file
Alternative: one VM	DigitalOcean Bangalore or AWS <code>ap-south-1</code> , ~₹500–900/mo	Run the same compose file. Simpler to reason about; one thing to keep alive

If you can afford one small VM, take it. Running the identical `docker-compose.yml` in production removes an entire class of "works on my machine" failure at the worst possible moment, and it lets you keep the graph in memory without cold starts.

The venue plan, which matters more than the cloud

Assume the wifi fails. Run the full stack locally on one laptop as the primary demo. Have the cloud deployment as the backup and the QR code for judges. Carry a phone hotspot as a third option. Pre-download every tile and dataset. If the demo requires internet to show a route, you have built a fragile demo — and Module 9 means you don't have to.

- Rehearse a `make demo-reset` that restores the database to a known state in seconds, so a second run in front of a second panel is clean.
- Seed one segment with a rainfall value that pushes it into `hazard`, so you can show the reroute on command instead of hoping the weather cooperates.
- Keep a recorded 90-second screen capture on the laptop. If everything fails, you still show the product.
- Bring an HDMI adapter, a USB-C hub, and a charged power bank. Every year a team loses on this.

PLAN Build order

Sequenced so that something demonstrable exists from day three onward. If you run out of time, you stop at whatever module you reached and it still shows.

Phase	Modules	Owner count	Done when
Days 1–2	0, 1, 2	All, then split	DB seeded, incident CSV compiled, corridor fixed
Days 3–4	3, 4	2 backend	Every segment has a score and an explanation
Day 5	5	1 backend	Three profiles diverge on a seeded hazard
Days 4–6	7, 8	1 backend + 2 frontend	Route screen works against the live API
Day 7	9	2 frontend	Airplane-mode test passes
Day 8	6, analytics	1	Deformation overlay + hotspot heatmap render
Day 9	Freeze	All	No new features. Bug fixes, deploy, rehearse
Day 10	Rehearse	All	Demo run three times, timed, on both laptop and cloud

PLAN The eight-minute demo

1. **0:00** — One sentence on the problem, on the corridor map, with the historical incident pins already visible. The pins do the arguing for you.
2. **1:00** — Plan Guwahati → Shillong. Show fastest. Show safest. Read the delta aloud: eighteen minutes for a materially lower risk exposure.
3. **2:30** — Tap the red segment. Show the five factors. This is the moment; slow down here.
4. **4:00** — Airplane mode. Plan a cached route. File a blockade report. Back online, watch it sync and the segment darken.
5. **6:00** — Deformation overlay and the precursor alert. State plainly that SAR processing is offline and scheduled.
6. **7:00** — Planner view: hotspot heatmap, and the one sentence about prioritising truck bays and bypasses from it.
7. **7:40** — Close on scale: same pipeline per corridor, offline-first by design, open data throughout.

Rehearse the narration to the second. A team that finishes calmly at 7:50 is more convincing than one that has more features and runs over.
